

The background of the slide is a photograph of the Mumbai skyline at night. The city's lights are reflected in the water of the harbor. In the foreground, there is a curved promenade with trees and streetlights on the left side.

Urban Blue Space Accessibility Analysis Using Arcpy Library: A Case Study On Mumbai, Maharashtra (2020)

Presented By:

Sreeja Bhaduri
PRN: 25070241065

Swapnanil Saha
PRN: 25070241071

Sritama Haldar
PRN: 25070241068

Tirsha Biswas
PRN: 25070241076

METHODOLOGICAL WORKFLOW

Urban Blue Space Accessibility Analysis using ArcPy Library:

A Case Study of Mumbai, India (2020) — Methodology Flowchart

1

Data Collection

Three datasets acquired via Google Earth Engine

- Maharashtra Boundary (Survey of India)
- Waterbodies — JRC Global Surface Water v1.4 (30m, Landsat)
- Population — WorldPop 2020, 100m resolution, UN-adjusted (University of Southampton)

2

Study Area Preparation

Select by Attribute + Dissolve | Tool: ArcPy

Mumbai City & Mumbai Suburban districts selected from Maharashtra boundary using SQL query → Both districts dissolved into one unified Mumbai boundary.

SelectLayerByAttribute

CopyFeatures

Dissolve

3

Population Clipping

Extract by Mask | Tool: ArcPy

WorldPop raster clipped precisely to Mumbai combined boundary using Extract by Mask to restrict analysis to study area.

ExtractByMask

4

Buffer Analysis

300m / 500m / 1000m | Tool: ArcPy

- 300m = High Accessibility (~5 min walk)
- 500m = Moderate Accessibility (~8 min walk)
- 1000m = Low Accessibility (~15 min walk)

arcpy.Buffer_analysis (dissolve_option=ALL)

arcpy.analysis.Clip

5

Zonal Statistics

Population Count per Buffer Zone | Tool: ArcPy

Total population (SUM) calculated within each buffer zone using clipped WorldPop raster as the value raster.

ZonalStatisticsAsTable (SUM)

6

Erase Analysis

Access vs No Access Zone | Tool: ArcPy

1000m buffer erased from Mumbai boundary → Remaining area shows underserved zones with no blue space within 1km.

arcpy.analysis.Erase

7

Euclidean Distance

Continuous Proximity Surface | Tool: ArcPy

Distance (metres) from every pixel to nearest waterbody calculated across full Mumbai extent. Range: 0m to 4446m. Cell size: 100m.

arcpy.sa.EucDistance

arcpy.sa.Int

8

Reclassification

4 Accessibility Classes | Tool: ArcPy

Class 1: Very High (0–300m) Class 2: High (300–1000m)
Class 3: Moderate (1000–3000m) Class 4: Low (3000–4446m)

arcpy.sa.Reclassify

arcpy.sa.RemapRange

DATA:

Mumbai Boundary Shapefile

ListFields(): Used to check Field Names within Maharashtra Shapefile.

SelectLayerByAttribute(): “Mumbai” and “Mumbai Suburban” districts are filtered and selected.

Dissolve(): Boundary between these two districts removed to create our study area Mumbai Shapefile.

Extracting Mumbai Boundary Shapefile Using Functions: SelectLayerByAttribute() & Dissolve()

INPUT CODE

```
# Extract Mumbai Combined Boundary (Mumbai City + Mumbai Suburban) from Maharashtra District Boundary Shapefile.

# Import Arcpy Library.
import arcpy

# Set the Workspace.
arcpy.env.workspace = "C:\\Urban Blue Space Accessibility\\Input"

# Input Maharashtra District Boundary.
Maharashtra = "C:\\Urban Blue Space Accessibility\\Input\\Maharashtra\\MAHARASHTRA_DISTRICT_BDY.shp" # Gathered from Survey of India Portal.

# Extract Field Names within Maharashtra District Boundary.
print("Fields within Maharashtra District Boundary Shapefile:")
for field in arcpy.ListFields(Maharashtra):
    print(" ", field.name)

# Extract District Names within Maharashtra District Boundary.
print("Districts within Maharashtra District Boundary Shapefile:")
with arcpy.da.SearchCursor(Maharashtra, ["DISTRICT"]) as cursor:
    for row in cursor:
        print(" ", row[0])

# Delete Search Cursor from Memory.
del cursor

# Create temporary Feature Layer from Maharashtra Feature Class.
arcpy.management.MakeFeatureLayer(Maharashtra, "District_Layer")

# Perform Select by Attribute (District): MUMBAI + MUMBAI SUBURBAN.
where_clause = "DISTRICT = 'MUMBAI' Or DISTRICT = 'MUMBAI SUBURBAN'"
arcpy.management.SelectLayerByAttribute("District_Layer", "NEW_SELECTION", where_clause)
print("Features selected:", arcpy.management.GetCount("District_Layer"))

# Save Layer as new Shapefile.
output_path = "C:\\Urban Blue Space Accessibility\\Input\\Mumbai_TwoDistricts.shp"
arcpy.management.CopyFeatures("District_Layer", output_path)
print("Mumbai City + Mumbai Suburban Districts saved.")

# Dissolve Layers into one Polygon.
in_features = "C:\\Urban Blue Space Accessibility\\Input\\Mumbai_TwoDistricts.shp"
out_feature_class = "C:\\Urban Blue Space Accessibility\\Input\\Mumbai_Boundary.shp"
arcpy.management.Dissolve(in_features, out_feature_class)
print("Dissolved Mumbai Boundary saved.")

# Projecting Mumbai_Boundary to UTM 43N.
in_features = "C:\\Urban Blue Space Accessibility\\Input\\Mumbai_Boundary.shp"
out_feature_class = "C:\\Urban Blue Space Accessibility\\Input\\Mumbai_Boundary_UTM.shp"
out_coordinate_system = arcpy.SpatialReference(32643)
arcpy.management.Project(in_features, out_feature_class, out_coordinate_system)
print("Projection set to UTM 43N.")
print("Done! Mumbai Boundary Shapefile is ready!")
```

V/S

OUTPUT

SINDHUDURG
SOLAPUR
MUMBAI SUBURBAN
THANE
DHARASHIV
WASHIM
WARDHA
YAVATMAL
AKOLA
AMRAVATI
BULDHANA
PARBHANI
JALNA
CHHATRAPATI SAMBHAJINAGAR

Features selected: 2

Mumbai City + Mumbai Suburban Districts saved.

Dissolved Mumbai Boundary saved.

Projection set to UTM 43N.

Done! Mumbai Boundary Shapefile is ready!

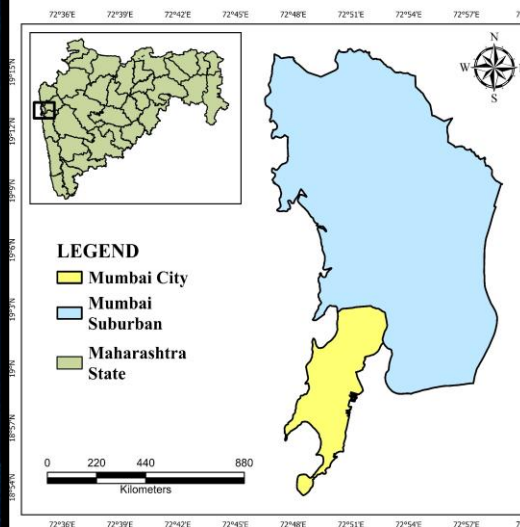
Fields within Maharashtra District Boundary Shapefile:

FID
Shape
OBJECTID
STATE_LGD
DISTRICT
STATE_UT
Shape_Leng
Shape_Area
Dist_LGD

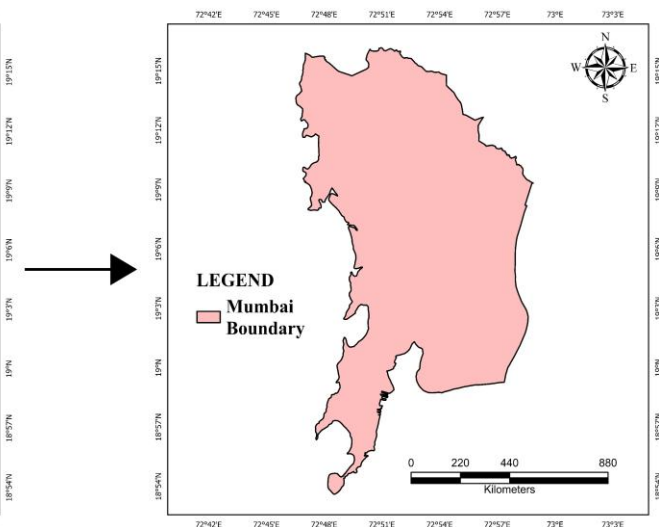
Districts within Maharashtra District Boundary Shapefile:

AHMEDNAGAR
BEED
BHANDARA
CHANDRAPUR
DHULE
GADCHIROLI
GONDIA
HINGOLI
JALGAON
KOLHAPUR
LATUR
MUMBAI

Selected Administrative Boundaries: Mumbai City & Mumbai Suburban
(Extracted from Maharashtra State)



Unified Mumbai Boundary Derived By:
Dissolving Mumbai City & Mumbai Suburban Districts



DATA:

Population Dataset

CheckOutExtension(): Used to ensure and check Spatial Analyst license is available.

ExtractByMask(): Extracts the Population Raster clipped to Mumbai Study Area Boundary.

Clipping Population Raster To Mumbai Boundary Using Function: ExtractByMask()

INPUT CODE

```
# Clip Population Raster to Mumbai Combined Boundary (MUMBAI CITY + MUMBAI SUBURBAN).

# Import Arcpy Library and ExtractByMask() from Spatial Analyst tool.
import arcpy
from arcpy.sa import ExtractByMask

# Set the Workspace.
arcpy.env.workspace = "C:\\Urban Blue Space Accessibility\\Input"

# Check out Spatial Analyst License.
arcpy.CheckOutExtension("Spatial")

# Add Input Layers: Population Raster + Mumbai Boundary.
population_raster = "C:\\Urban Blue Space Accessibility\\Input\\Mumbai_Population_100m_2020 (2).tif" # WorldPop Global Population Dataset (100m)
mumbai_boundary = "C:\\Urban Blue Space Accessibility\\Input\\Mumbai_Boundary_UTM.shp"

# Clip Population Raster to Mumbai Boundary using ExtractByMask().
print("Clipping Population Raster to Mumbai Boundary.....")
extracted_population = ExtractByMask(population_raster, mumbai_boundary)

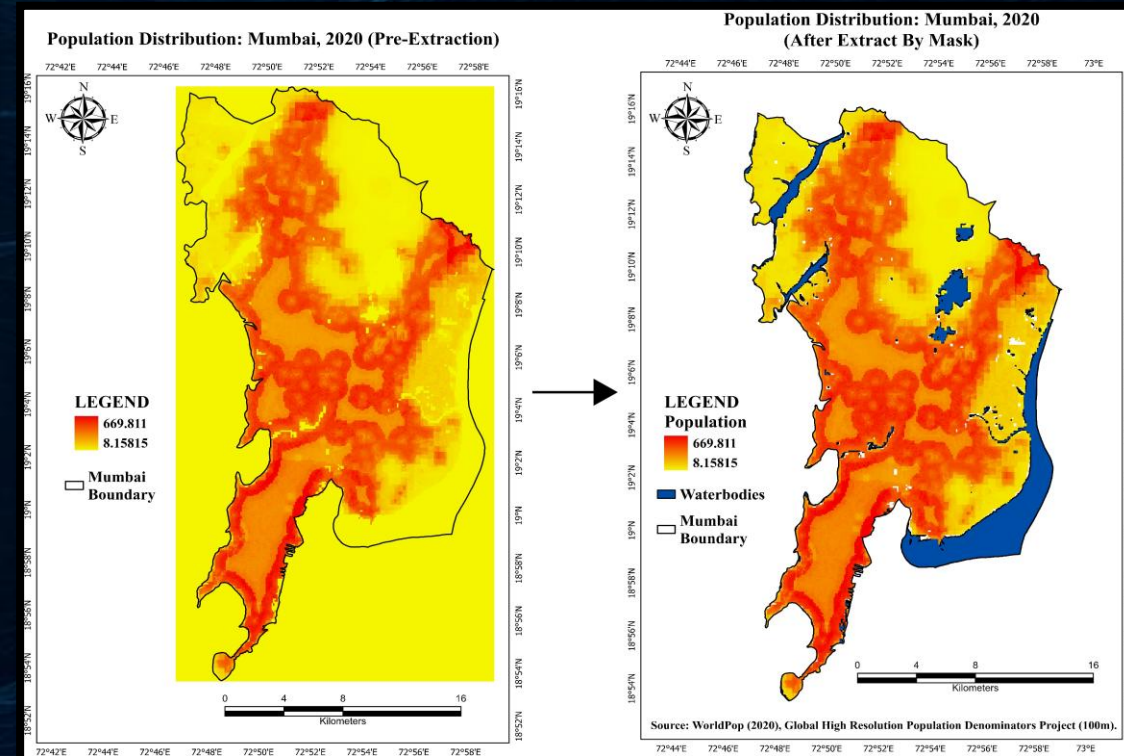
# Save the Output.
extracted_population.save("C:\\Urban Blue Space Accessibility\\Input\\Mumbai_Population_UTM_Clippped.tif")
print("Done! Clipped Raster saved as Mumbai_Population_UTM_Clippped.tif")

Clipping Population Raster to Mumbai Boundary.....
Done! Clipped Raster saved as Mumbai_Population_UTM_Clippped.tif
```

Highest Population Densities (in Deep Red) are concentrated in the Western and Southern peninsular areas, notably Dharavi, Bandra and South Mumbai.

The WorldPop (100 m) Population dataset was clipped to the Mumbai boundary using ExtractByMask(), eliminating surrounding area noise and confining population values to the study extent.

OUTPUT

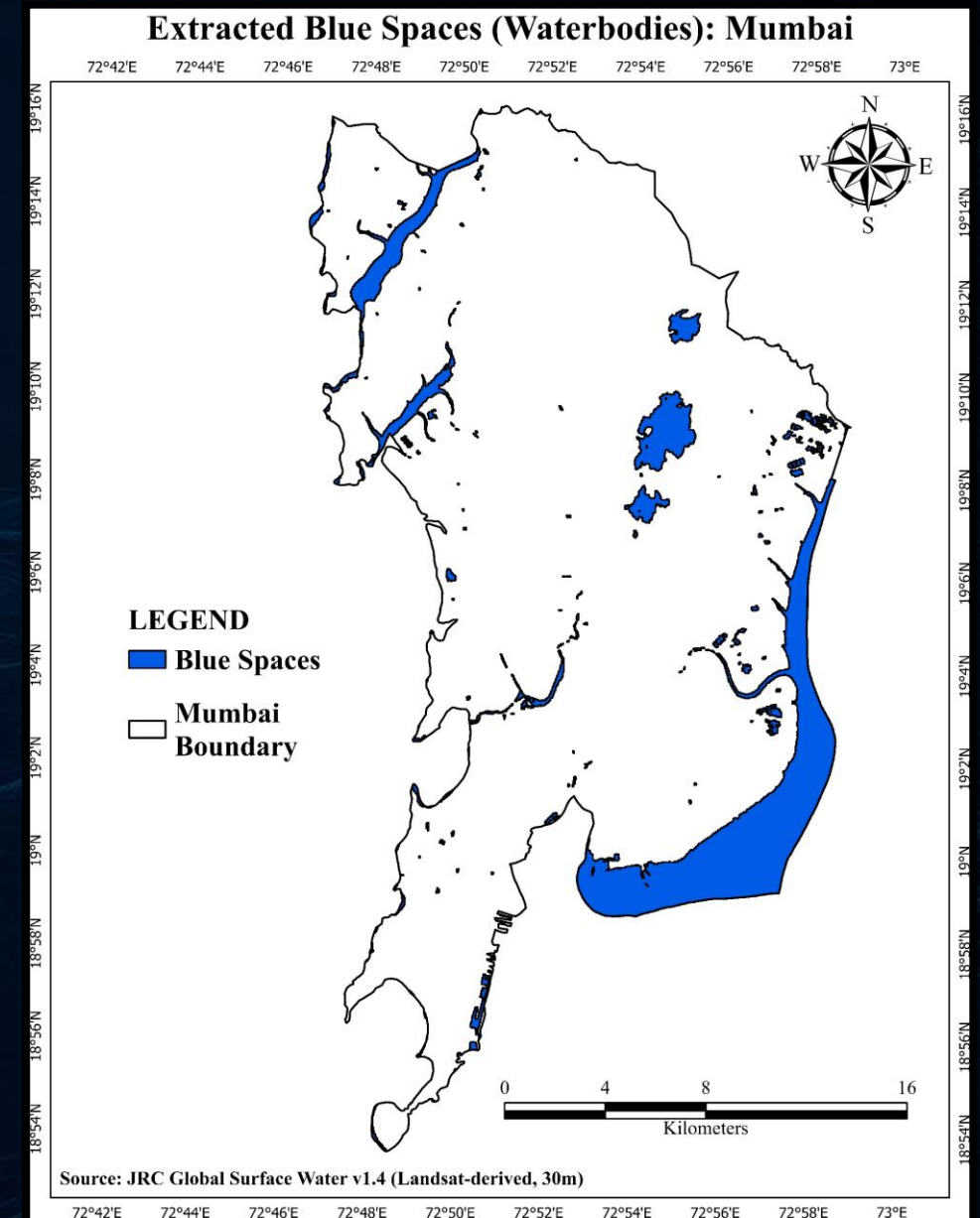


DATA:

Waterbodies Extraction and Buffer Analysis

Buffer_analysis(): This is used to create Buffer Zones around vectors (here, waterbodies polygons).

Clip(): This clips each Buffer to the Study Area Boundary.



INPUT CODE

```
# Buffer Analysis: 300m, 500m, 1000m - Depicts which areas fall within walking distance of Waterbodies.

# Import Arcpy Library.
import arcpy

# Set the Workspace.
arcpy.env.workspace = "C:\\Urban Blue Space Accessibility\\Input"

# Add Input Layers: Waterbodies and Mumbai_Boundary Shapefiles.
waterbodies = "C:\\Urban Blue Space Accessibility\\Input\\Waterbodies_UTM.shp" # JRC Global Surface Water - Monthly Water History v1.4 Dataset (30m)
mumbai_boundary = "C:\\Urban Blue Space Accessibility\\Input\\Mumbai_Boundary_UTM.shp"

# Buffer: 300m (High Accessibility).
in_feature = waterbodies
out_feature_class = "C:\\Urban Blue Space Accessibility\\Output\\Buffer_300m.shp"
arcpy.Buffer_analysis(in_feature, out_feature_class, "300 Meters", dissolve_option = "ALL")
print("300m Buffer Done")

# Buffer: 500m (Medium Accessibility).
in_feature = waterbodies
out_feature_class = "C:\\Urban Blue Space Accessibility\\Output\\Buffer_500m.shp"
arcpy.Buffer_analysis(in_feature, out_feature_class, "500 Meters", dissolve_option = "ALL")
print("500m Buffer Done")

# Buffer: 1000m (Low Accessibility).
in_feature = waterbodies
out_feature_class = "C:\\Urban Blue Space Accessibility\\Output\\Buffer_1000m.shp"
arcpy.Buffer_analysis(in_feature, out_feature_class, "1000 Meters", dissolve_option = "ALL")
print("1000m Buffer Done")

# Clip Buffers to Mumbai_Boundary.

# 300m Buffer.
in_features = "C:\\Urban Blue Space Accessibility\\Output\\Buffer_300m.shp"
clip_features = mumbai_boundary
out_feature_class = "C:\\Urban Blue Space Accessibility\\Output\\Buffer_300m_Clippped.shp"
arcpy.analysis.Clip(in_features, clip_features, out_feature_class)

# 500m Buffer.
in_features = "C:\\Urban Blue Space Accessibility\\Output\\Buffer_500m.shp"
clip_features = mumbai_boundary
out_feature_class = "C:\\Urban Blue Space Accessibility\\Output\\Buffer_500m_Clippped.shp"
arcpy.analysis.Clip(in_features, clip_features, out_feature_class)

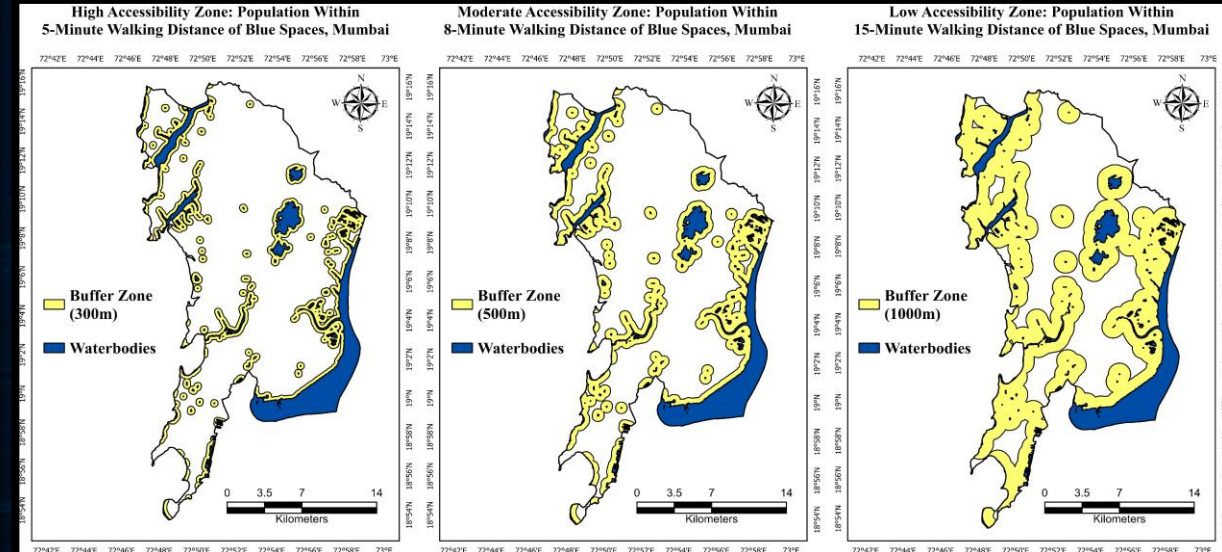
# 1000m Buffer.
in_features = "C:\\Urban Blue Space Accessibility\\Output\\Buffer_1000m.shp"
clip_features = mumbai_boundary
out_feature_class = "C:\\Urban Blue Space Accessibility\\Output\\Buffer_1000m_Clippped.shp"
arcpy.analysis.Clip(in_features, clip_features, out_feature_class)

# Display Message.
print("All Buffers clipped to Mumbai Boundary.")
print("Buffer Analysis Completed!")

300m Buffer Done
500m Buffer Done
1000m Buffer Done
All Buffers clipped to Mumbai Boundary.
Buffer Analysis Completed!
```

Creating Buffer Zones Around Extracted Waterbodies Using Functions: Buffer_analysis() & Clip()

OUTPUT



300 m — High Access

5-min walking distance: Captures population with immediate Blue Space Proximity.

500 m — Moderate Access

8-min walk: Intermediate Zone that expands spatial coverage significantly.

1000 m — Low Access

15-min walk: The largest zone, covering most of Mumbai's periphery and suburban areas.

Zonal Statistics From Population Dataset

CheckOutExtension(): Used to ensure and check Spatial Analyst license is available.

ZonalStatisticsAsTable(): Used to calculate statistics of whatever attribute is selected (here, Buffer Zones & Population Raster).

Assessing Population Within Each Buffer Zone Using Function: ZonalStatisticsAsTable()

INPUT CODE

```
# Zonal Statistics Analysis: Population Living within each Buffer Zone.
# Import Arcpy Library and ZonalStatisticsAsTable() from Spatial Analyst tool.
import arcpy
from arcpy.sa import ZonalStatisticsAsTable

# Set the Workspace.
arcpy.env.workspace = "C:\\Urban Blue Space Accessibility\\Input"
# Check out Spatial Analyst License.
arcpy.CheckOutExtension("Spatial")

# Add Input Layers: Population Raster + Buffer Vectors (300m, 500m, 1000m).
population_raster = "C:\\Urban Blue Space Accessibility\\Input\\Mumbai_Population_UTM_Clippped.tif"
buffer_300 = "C:\\Urban Blue Space Accessibility\\Output\\Buffer_300m_Clippped.shp"
buffer_500 = "C:\\Urban Blue Space Accessibility\\Output\\Buffer_500m_Clippped.shp"
buffer_1000 = "C:\\Urban Blue Space Accessibility\\Output\\Buffer_1000m_Clippped.shp"

# Calculate Population within 300m Buffer.
in_zone_data = buffer_300
zone_field = "FID"
in_value_raster = population_raster
out_table = "C:\\Urban Blue Space Accessibility\\Output\\ZS_300m.dbf"
ZonalStatisticsAsTable(in_zone_data, zone_field, in_value_raster, out_table, ignore_nodata = "DATA", statistics_type = "SUM")
print("Population within 300m Buffer calculated!")

# Calculate Population within 500m Buffer.
in_zone_data = buffer_500
zone_field = "FID"
in_value_raster = population_raster
out_table = "C:\\Urban Blue Space Accessibility\\Output\\ZS_500m.dbf"
ZonalStatisticsAsTable(in_zone_data, zone_field, in_value_raster, out_table, ignore_nodata = "DATA", statistics_type = "SUM")
print("Population within 500m Buffer calculated!")
```

```
# Calculate Population within 1000m Buffer.
in_zone_data = buffer_1000
zone_field = "FID"
in_value_raster = population_raster
out_table = "C:\\Urban Blue Space Accessibility\\Output\\ZS_1000m.dbf"
ZonalStatisticsAsTable(in_zone_data, zone_field, in_value_raster, out_table, ignore_nodata = "DATA", statistics_type = "SUM")
print("Population within 1000m Buffer calculated!")

# Display Results.

# For 300m Buffer.
print("Population within Blue Space Access:")
with arcpy.da.SearchCursor("C:\\Urban Blue Space Accessibility\\Output\\ZS_300m.dbf", ["SUM"]) as cursor:
    for row in cursor:
        print("Within 300m Buffer (High Access):", row[0])

# For 500m Buffer.
with arcpy.da.SearchCursor("C:\\Urban Blue Space Accessibility\\Output\\ZS_500m.dbf", ["SUM"]) as cursor:
    for row in cursor:
        print("Within 500m Buffer (Medium Access):", row[0])

# For 1000m Buffer.
with arcpy.da.SearchCursor("C:\\Urban Blue Space Accessibility\\Output\\ZS_1000m.dbf", ["SUM"]) as cursor:
    for row in cursor:
        print("Within 1000m Buffer (Low Access):", row[0])
```

OUTPUT

```
Population within 300m Buffer calculated!
Population within 500m Buffer calculated!
Population within 1000m Buffer calculated!
Population within Blue Space Access:
Within 300m Buffer (High Access): 1535973.80546
Within 500m Buffer (Medium Access): 2829381.34203
Within 1000m Buffer (Low Access): 6031259.16932
```


Access and No Access Zones Using Erase Analysis

Erase(): Used to create a new Shapefile by removing areas present within another Shapefile and keeping only excluded areas (here, No Blue Space Access Zones).

Creating Access and No Access Zones Using Function: Erase()

INPUT CODE

V/S

OUTPUT

```
# Erase Analysis: Create a new Shapefile with "No Access Zones": Area beyond 1000m Buffer.
```

```
# Import Arcpy Library.
```

```
import arcpy
```

```
# Set the Workspace.
```

```
arcpy.env.workspace = "C:\\Urban Blue Space Accessibility\\Input"
```

```
# Add Input Layers: Mumbai_Boundary, 1000m Buffer.
```

```
mumbai_boundary = "C:\\Urban Blue Space Accessibility\\Input\\Mumbai_Boundary_UTM.shp"
```

```
buffer_1000 = "C:\\Urban Blue Space Accessibility\\Output\\Buffer_1000m_Clipped.shp"
```

```
# Using Erase() to create "No Access Zone" Shapefile.
```

```
in_features = mumbai_boundary
```

```
erase_features = buffer_1000
```

```
out_feature_class = "C:\\Urban Blue Space Accessibility\\Output\\No_Access_Zone.shp"
```

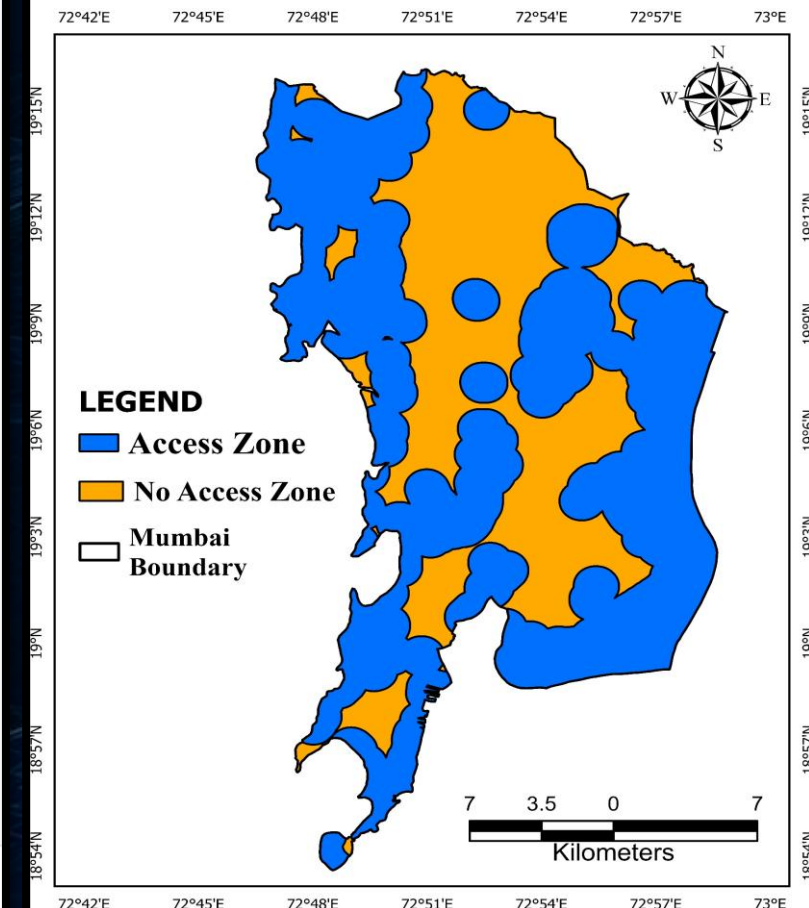
```
arcpy.analysis.Erase(in_features, erase_features, out_feature_class)
```

```
# Display Message.
```

```
print("No Access Zone Shapefile created!")
```

```
No Access Zone Shapefile created!
```

Access and No Access Zones to Blue Spaces: Based on 1000m Threshold, Mumbai



Euclidean Distance Analysis

CheckOutExtension(): Used to ensure and check Spatial Analyst license is available.

EucDistance(): Calculates straight-line distances from Waterbodies to each Cell within Mumbai Boundary Shapefile.

Int(): Converts the Euclidean Distance Raster to Integer Format (without Decimal Places).

Straight Line Distance From Nearest Waterbody Using Function: EucDistance()

INPUT CODE

```
# Euclidean Distance Analysis: Create a Raster showing distance (in Meters) from every pixel to the nearest Waterbody.

# Import Arcpy Library and EucDistance(), Int() from Spatial Analyst tool.
import arcpy
from arcpy.sa import EucDistance, Int

# Set the Workspace.
arcpy.env.workspace = "C:\\Urban Blue Space Accessibility\\Input"

# Check out Spatial Analyst License.
arcpy.CheckOutExtension("Spatial")

# Add Input Layers: Waterbodies + Mumbai Boundary.
waterbodies = "C:\\Urban Blue Space Accessibility\\Input\\Waterbodies_UTM.shp"
mumbai_boundary = "C:\\Urban Blue Space Accessibility\\Input\\Mumbai_Boundary_UTM.shp"

# Set the Mask: Mumbai_Boundary.
arcpy.env.mask = mumbai_boundary
arcpy.env.extent = mumbai_boundary

# Euclidean Distance Analysis.
source = waterbodies
euclidean_distance = EucDistance(source, cell_size = 100)

# Save the Output.
euclidean_distance_int = Int(euclidean_distance)
euclidean_distance_int.save("C:\\Urban Blue Space Accessibility\\Output\\Euclidean_Distance.tif")

# Display Message.
print("Euclidean Distance Raster saved!")

Euclidean Distance Raster saved!
```

OUTPUT



Reclassification Of Euclidean Distance Into Very High, High, Medium, Low Zones

RemapRange(): Used to specify the size of each Class to be created (including Highest and Lowest value for computing Ranges).

Reclassify(): Used to create the reclassified Raster by using the Classes specified as above.

Int(): Converts the Euclidean Distance Raster to Integer Format (without Decimal Places).

Accessibility To Urban Blue Spaces Analysis Using Functions: RemapRange() & Reclassify()

INPUT CODE

```
# Reclassify Euclidean Distance raster into four classes: Very High, High, Medium and Low Access.

# Import Arcpy Library and Reclassify(), RemapRange(), Int() from Spatial Analyst tool.
import arcpy
from arcpy.sa import Reclassify, RemapRange

# Set the Workspace.
arcpy.env.workspace = "C:\\Urban Blue Space Accessibility\\Input"

# Check out Spatial Analyst License.
arcpy.CheckOutExtension("Spatial")

# Input Layer: Euclidean Distance.
euclidean_distance = "C:\\Urban Blue Space Accessibility\\Output\\Euclidean_Distance.tif"

# Set Classification Classes: Euclidean Distance Raster.
# 0-500m: Very High Access, 500-1000m: High Access, 1000-2000m: Medium Access, >2000m: Low Access.
reclass_rules = RemapRange([[0, 500, 1], [500, 1000, 2], [1000, 2000, 3], [2000, 4500, 4]])

# Reclassify Euclidean Distance Raster.
classes = Reclassify(euclidean_distance, reclass_field = "Value", remap = reclass_rules)

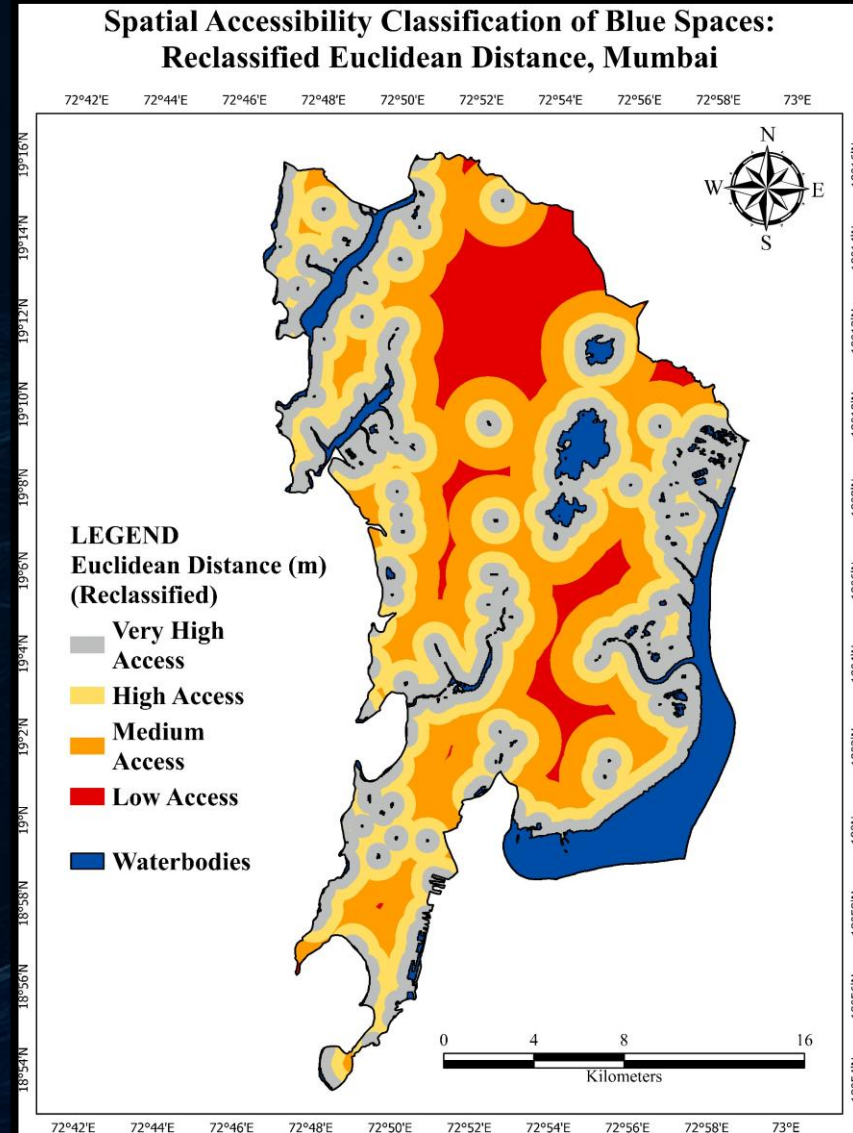
# Convert to Integer.
classes_int = Int(classes)

# Save the Output.
classes_int.save("C:\\Urban Blue Space Accessibility\\Output\\Accessibility_Classes_Int.tif")

# Display Messages.
print("Reclassified Raster saved!")
print("Class 1 = Very High Access (within 500m)")
print("Class 2 = High Access (500m - 1000m)")
print("Class 3 = Medium Access (1000m - 2000m)")
print("Class 4 = Low Access (Beyond 2000m)")

Reclassified Raster saved!
Class 1 = Very High Access (within 500m)
Class 2 = High Access (500m - 1000m)
Class 3 = Medium Access (1000m - 2000m)
Class 4 = Low Access (Beyond 2000m)
```

OUTPUT



Findings and Conclusion



Very High (0–300m)

Immediately adjacent to waterbodies; highest direct access.



High (300–1000m)

Covers large portions of the periphery and coastal belts.



Medium (1000–2000m)

Intermediate zone; moderate walking effort required.



Low / No Access (>2000m)

Northeastern suburban core; highest priority for interventions.

***THANK
YOU***